

Navegação no ROS 2

Nav2

Walter Fetter Lages

`w.fetter@ieee.org`

Universidade Federal do Rio Grande do Sul

Escola de Engenharia

Departamento de Sistemas Elétricos de Automação e Energia

CCA99006 Tópicos Especiais em Engenharia de Controle e Automação II



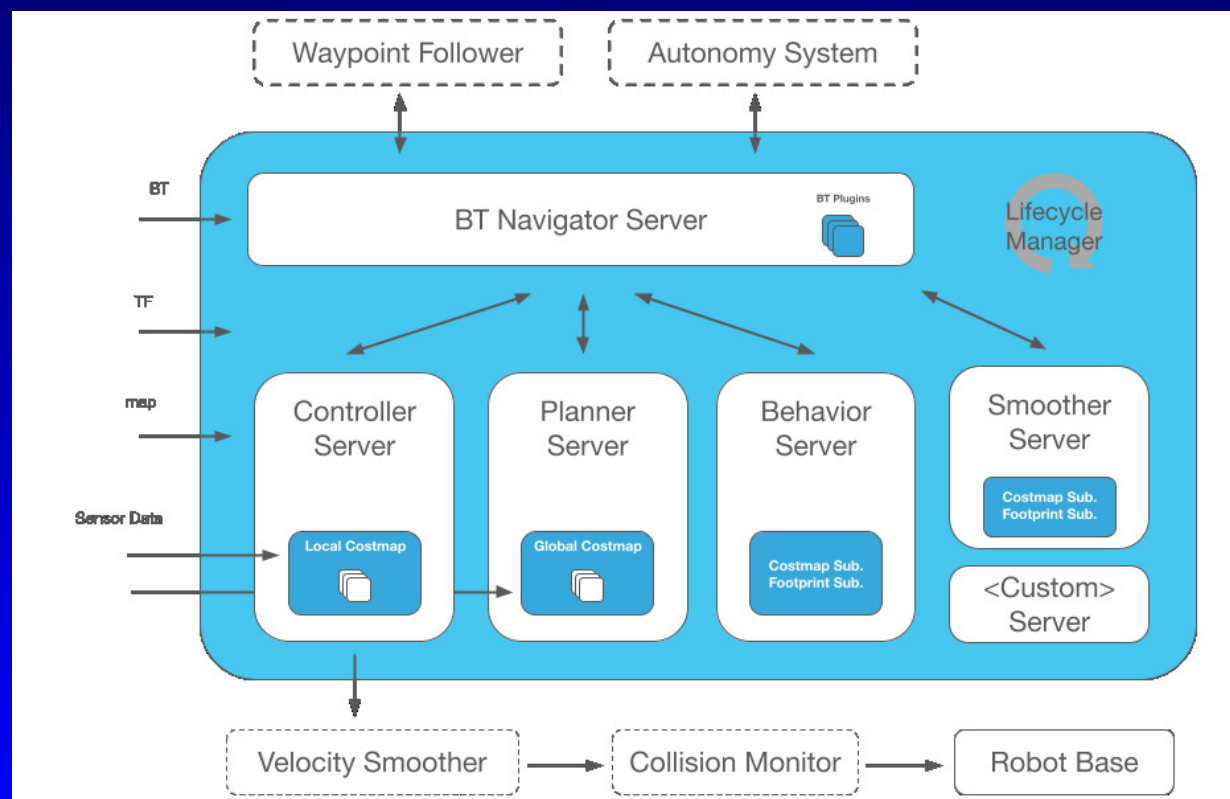


Introdução

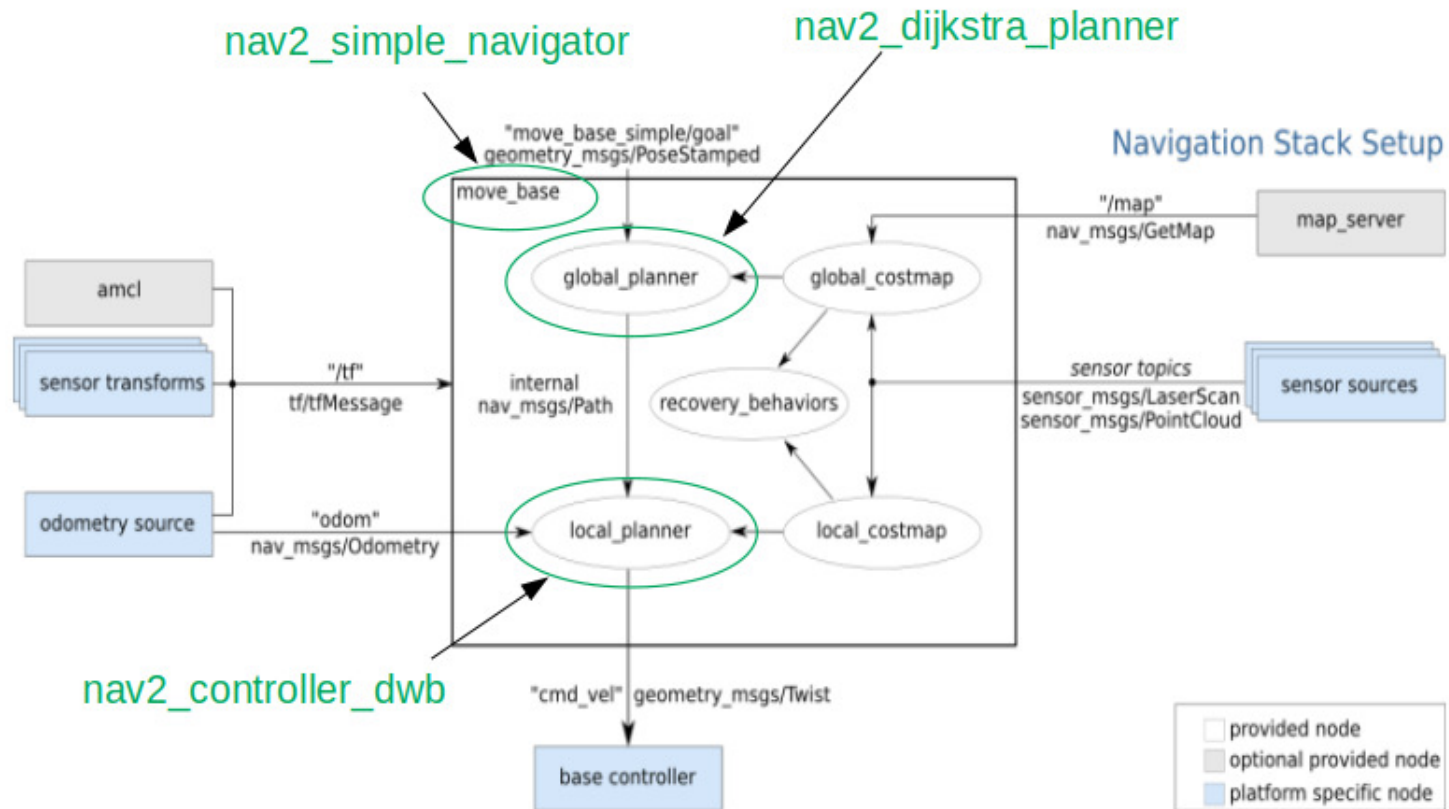
- Nav2 é o sucessor do *stack* de navegação do ROS
- Funcionalidades para mover o robô do ponto A ao ponto B em um ambiente real
 - Planejamento de caminhos
 - Cálculo das referências para os controladores
 - Desvio de obstáculos
 - Recuperação de falhas
- Faz a integração entre
 - Controle
 - Localização
 - Controle

Nav2

- Usa árvores de comportamento para chamar servidores de ação
- Servidores de ação calculam caminhos, referências para controladores, etc
- Cada ação é provida por um nodo separado



Comparação com ROS 1





Entradas e Saída do Nav2

- Entradas
 - Transformações TF (tópico / $t f$)
 - Mapa ou Mapas
 - Árvore de comportamento
 - Dados de sensores
- Saída
 - Comandos de velocidade
 - Omnidirecional
 - Diferencial
 - Pernas
 - Ackermann

Ferramentas do Nav2

- Carga, serviço e armazenamento de mapas: `map_server`
- Localização: `AMCL`
- Planejamento de caminho: `planner_server`
- Controle de rastreamento do caminho: `controller_server`
- Suavização e factibilidade de caminhos: `Smoother`
- Conversão de dados de sensores em mapas de custo: `Costmap 2D`
- Construção de comportamentos: `bt_navigator`
- Comportamentos de recuperação: `recoveries_server`
- Rastreamento de pontos: `waypoint_follower`
- Gerenciamento de ciclo de vida e *watchdog*:
`lifecycle_manager`
- *Plugins* para outros algoritmos: `Nav2 Core`



Conceitos

- Servidor de ação
- Nodos gerenciados
- Árvores de comportamento
- Servidores de navegação
 - Planejadores
 - Controladores (cinemáticos)
 - Recuperação
 - Pontos de passagem

Conceitos

- Estimação de estado
- $\text{map} \longrightarrow \text{odom} \longrightarrow \text{base_link} \longrightarrow [\text{sensor_frames}]$
- Odometria: publica a transformação
 $\text{odom} \longrightarrow \text{base_link}$
- Localização: publica a transformação
 $\text{map} \longrightarrow \text{odom}$
- Representação do ambiente
 - Mapas de custo
 - *Layers*
 - Filtros



Instalação do Nav2

sudo apt update

sudo apt install ros-\$ROS_DISTRO-navigation2

source ~/colcon_ws/install/setup.bash

Pacote `twil_2dnav`

```
twil_2dnav/  
├─ CMakeLists.txt  
├─ config/  
│   ├── bt_navigator_params.yaml  
│   ├── controller_params.yaml  
│   ├── planner_params.yaml  
│   └── recoveries_params.yaml  
├─ launch/  
│   ├── gazebo.launch.xml  
│   └── nav2_navigator.launch.xml  
├─ package.xml  
├─ rviz/  
│   └── twil_2dnav.rviz
```

gazebo.launch.xml

<launch>

<arg name="pause" default="true"/>

<arg name="gui" default="true"/>

<arg name="use_sim_time" default="true"/>

<arg name="wam" default="false"/>

<arg name="gps" default="false"/>

**<arg name="world" default="\$ (find-pkg-share ufrgs_gazebo)/worlds/
centenario.world"/>**

<arg name="controller" default="twist_mrac_linearizing_controller"/>

**<arg name="config" default="\$ (find-pkg-share twil_bringup)/config/\$(
var controller).yaml"/>**

gazebo.launch.xml

```
<include file="$(find-twil-bringup)/launch/gazebo.launch.xml"
>
  <arg name="pause" value="$(var pause)"/>
  <arg name="gui" value="$(var gui)"/>
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="wam" value="$(var wam)"/>
  <arg name="d435" value="false"/>
  <arg name="gps" value="$(var gps)"/>
  <arg name="world" value="$(var world)"/>
  <arg name="controller" value="$(var controller)"/>
  <arg name="config" value="$(var config)"/>
</include>
```



gazebo.launch.xml

```
<include file="$(find-twil_2dnav)/launch/nav2_navigator.  
  launch.xml">
```

```
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
```

```
  <arg name="controller" value="$(var controller)"/>
```

```
</include>
```

```
<node name="rviz" pkg="rviz2" exec="rviz2" args="-d $(find-twil_2dnav)/rviz/twil_2dnav.rviz">
```

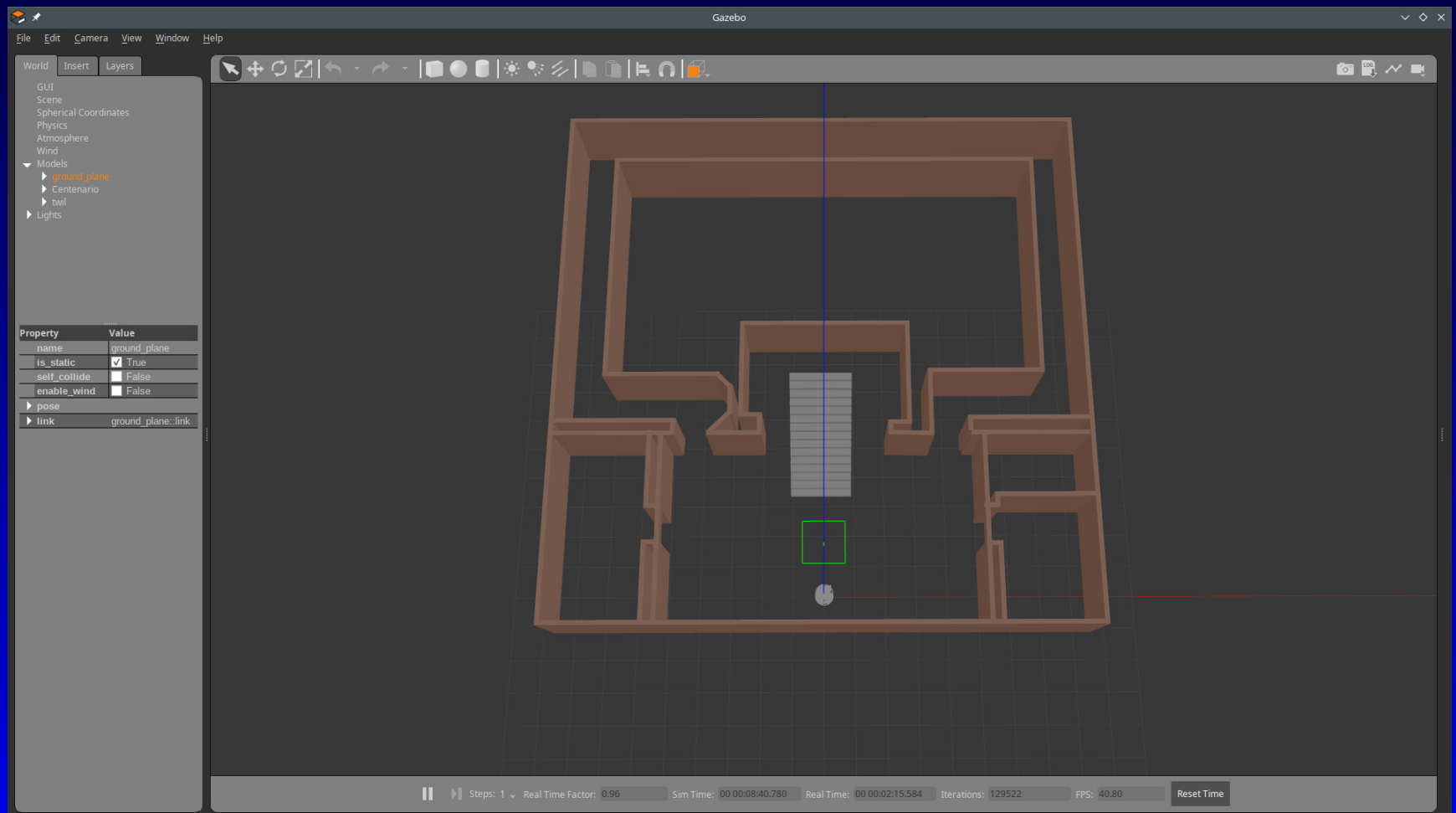
```
  <param name="use_sim_time" value="$(var use_sim_time)"/>
```

```
</node>
```

```
</launch>
```

Gazebo com *World* Centenário

```
ros2 launch twil_2dnav gazebo.launch.xml
```





nav2_navigator.launch.xml

<launch>

<arg name="use_sim_time" default="false"/>

**<arg name="map" default="\$(find-pkg-share ufrgs_maps)/map/
centenario1.yaml"/>**

<arg name="autostart" default="true"/>

<arg name="controller" default=""/>

<node name="map_server" pkg="nav2_map_server" exec="map_server">

<param name="use_sim_time" value="\$(var use_sim_time)"/>

<param name="yaml_filename" value="\$(var map)"/>

</node>

<node pkg="tf2_ros" exec="static_transform_publisher" name="odom_frame_publisher" args="0 0 0 0 0 0 1 map odom"/>



nav2_navigator.launch.xml

```
<node pkg="nav2_controller" exec="controller_server" name="
  controller_server">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param from="$(find-pkg-share twil_2dnav)/config/controller_params.
    yaml"/>
  <remap from="odom" to="$(var controller)/odom"/>
  <remap from="cmd_vel" to="$(var controller)/command"/>
</node>

<node pkg="nav2_planner" exec="planner_server" name="planner_server
  ">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param from="$(find-pkg-share twil_2dnav)/config/planner_params.
    yaml"/>
</node>
```



nav2_navigator.launch.xml

```
<node pkg="nav2_recoveries" exec="recoveries_server" name="
  recoveries_server">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param from="$(find-pkg-share twil_2dnave)/config/recoveries_params.
    yaml"/>
  <param name="robot_base_frame" value="twil_origin"/>
  <remap from="cmd_vel" to="$(var controller)/command"/>
</node>

<node pkg="nav2_bt_navigator" exec="bt_navigator" name="bt_navigator
  ">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param from="$(find-pkg-share twil_2dnave)/config/
    bt_navigator_params.yaml"/>
  <param name="robot_base_frame" value="twil_origin"/>
  <remap from="odom" to="$(var controller)/odom"/>
</node>
```



nav2_navigator.launch.xml

```
<node pkg="nav2_lifecycle_manager" exec="lifecycle_manager" name="
  lifecycle_manager_nav2">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="autostart" value="$(var autostart)"/>
  <param name="node_names" value="['map_server',
    'controller_server',
    'planner_server',
    'recoveries_server',
    'bt_navigator']"/>
</node>

</launch>
```

map_server

- Publica os mapas
- É um nodo gerenciado
 - Pode estar em um dos estados: Unconfigured, Inactive, Active, Finalized
 - Transição por linha de comando

```
ros2 lifecycle set /map_server configure
```

```
ros2 lifecycle set /map_server activate
```

- Transição por gerenciador

```
<node pkg="nav2_lifecycle_manager" exec="
  lifecycle_manager" name="lifecyle_manager">
  <param name="autostart" value="true"/>
  <param name="node_names" value="['map_server']"/>
</node>
```

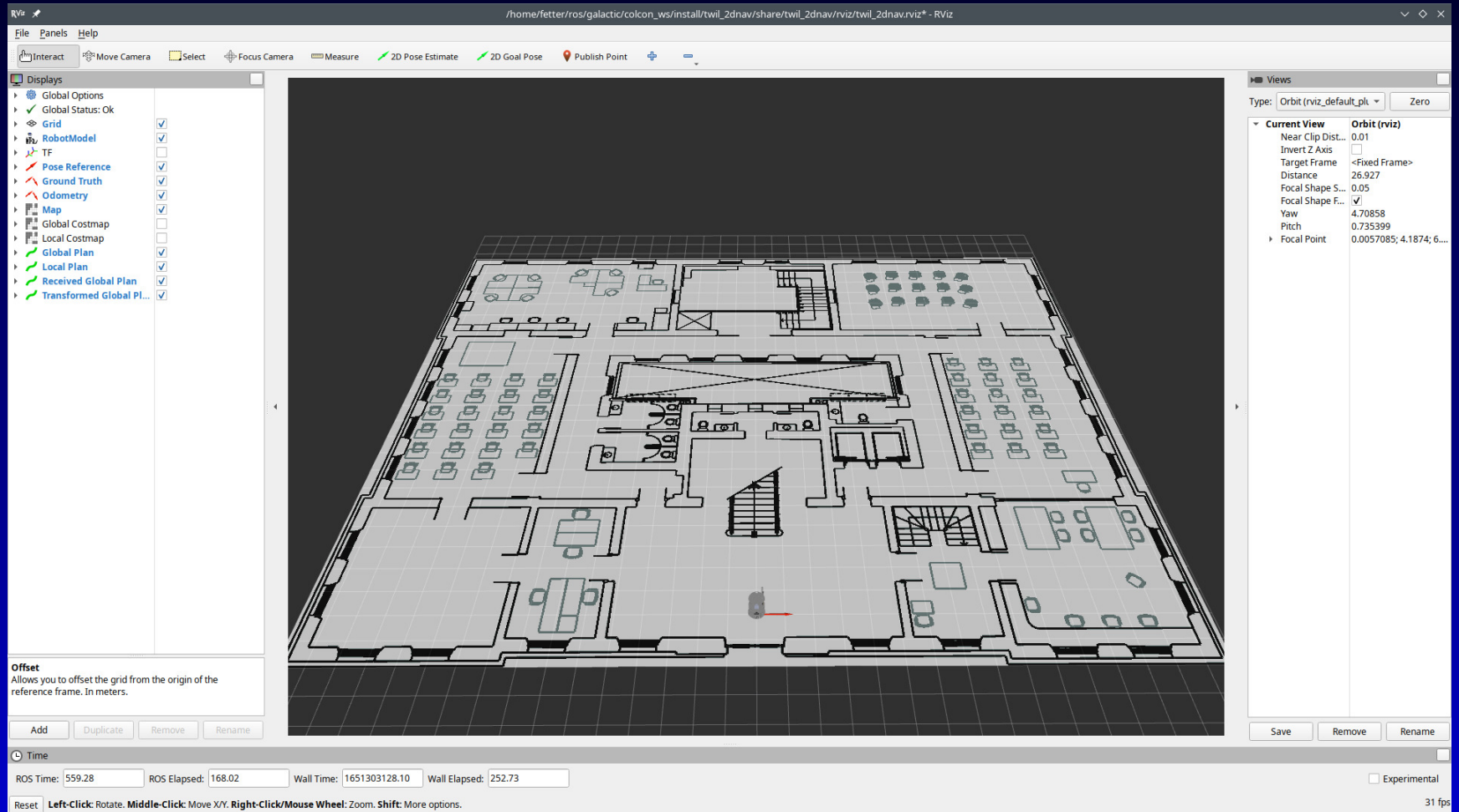


centenario1.yaml

image: centenario1.pgm
resolution: 0.01
origin: [-16.0, -2.0, 0.0]
occupied_thresh: 0.7
free_thresh: 0.3
negate: 0



Mapa do Ambiente



Costmap 2D

- Representação do ambiente na forma de uma grade ocupação
 - Cada célula tem um valor entre 0 (livre) e 254 (ocupada)
 - Valores intermediários representam incerteza
- Algoritmos de navegação afastam o robô de obstáculos como se fosse um campo de potencial
- Implementado no pacote `nav2_costmap_2d`



Camadas do nav2_costmap_2d

- Cada camada implementa uma função que contribui para o custo total de cada célula
- *static layer*: custo estático obtido do mapa
- *inflation layer*: custo adicionado em torno de obstáculos devido à geometria do robô
- *range layer*: sonar e infravermelho
- *obstacle layer*: obstáculos detectados pelo sensores
- *voxel layer*: obstáculos 3D

Planner Server

- Faz o planejamento global até o destino
- Gera o mapa de custo global
- *Plugins* para diversos algoritmos
 - NavFn: usa A^* ou Dijkstra
 - Smac 2D: A^* 2D usando 4 ou 8 vizinhos com suavização e multiresolução
 - Theta*: Usa linha de visada para gerar segmentos de caminho não discretizados
 - Smac Hybrid- A^* : Considere ao raio mínimo de curva do robô
 - Smac Lattice: Garante que o caminho respeita as restrições cinemáticas do robô



planner_params.yaml

planner_server:

ros__parameters:

expected_planner_frequency: 20.0

planner_plugins: ["GridBased"]

GridBased:

plugin: "nav2_navfn_planner/NavfnPlanner"

tolerance: 0.5

use_astar: false

allow_unknown: true

global_costmap:

global_costmap:

ros__parameters:

update_frequency: 1.0

publish_frequency: 1.0

global_frame: map

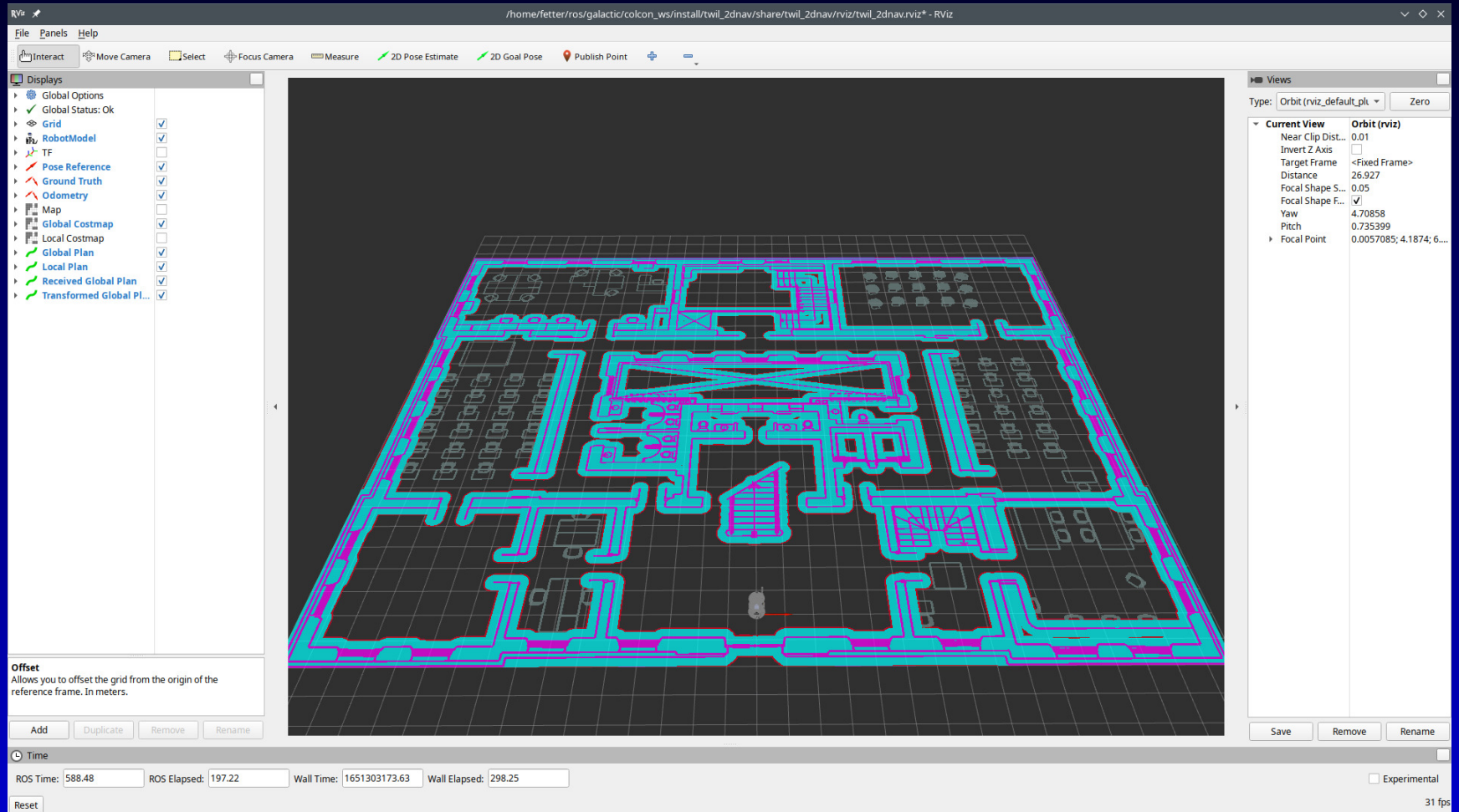


planner_params.yaml

```
robot_base_frame: twil_origin
robot_radius: 0.3
resolution: 0.05
track_unknown_space: true
plugins: ["static_layer", "inflation_layer"]
static_layer:
  plugin: "nav2_costmap_2d::StaticLayer"
  map_subscribe_transient_local: True
inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  cost_scaling_factor: 3.0
  inflation_radius: 0.35
always_send_full_costmap: True
```



Mapa de Custo Global



Controller Server

- Faz o planejamento local gerando a referência de velocidade
- Gera o mapa de custo local
- *Plugins* para diversos algoritmos
 - *Dynamic Window Approach - B (DWB)*: Gera trajetórias possíveis em função da aceleração e dessaceleração máximas e distância aos obstáculos
 - *Plugins* para atribuir custos baseados em diversas situações críticas
 - *Timed Elastic Band (TEB)*: MPC
 - *Regulated Pure Pursuit*: Velocidade linear selecionada heurísticamente e calcula a velocidade angular



controller_params.yaml

controller_server:

ros__parameters:

controller_frequency: 20.0

min_x_velocity_threshold: 0.001

min_y_velocity_threshold: 0.5

min_theta_velocity_threshold: 0.001

failure_tolerance: 0.3

progress_checker_plugin: "progress_checker"

goal_checker_plugins: ["general_goal_checker"]

controller_plugins: ["FollowPath"]

Progress checker parameters

progress_checker:

plugin: "nav2_controller::SimpleProgressChecker"

required_movement_radius: 0.5

movement_time_allowance: 10.0

controller_params.yaml

```
general_goal_checker:  
  stateful: True  
  plugin: "nav2_controller::SimpleGoalChecker"  
  xy_goal_tolerance: 0.25  
  yaw_goal_tolerance: 0.25  
# DWB parameters  
FollowPath:  
  plugin: "dwb_core::DWBLocalPlanner"  
  debug_trajectory_details: True  
  min_vel_x: 0.0  
  min_vel_y: 0.0  
  max_vel_x: 0.26  
  max_vel_y: 0.0  
  max_vel_theta: 1.0  
  min_speed_xy: 0.0  
  max_speed_xy: 0.26  
  min_speed_theta: 0.0
```



controller_params.yaml

```
acc_lim_x: 2.5
acc_lim_y: 0.0
acc_lim_theta: 3.2
decel_lim_x: -2.5
decel_lim_y: 0.0
decel_lim_theta: -3.2
vx_samples: 20
vy_samples: 5
vtheta_samples: 20
sim_time: 1.7
linear_granularity: 0.05
angular_granularity: 0.025
transform_tolerance: 0.2
xy_goal_tolerance: 0.25
trans_stopped_velocity: 0.25
short_circuit_trajectory_evaluation: True
stateful: True
```



controller_params.yaml

```
critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
```

```
BaseObstacle.scale: 0.02
```

```
PathAlign.scale: 32.0
```

```
PathAlign.forward_point_distance: 0.1
```

```
GoalAlign.scale: 24.0
```

```
GoalAlign.forward_point_distance: 0.1
```

```
PathDist.scale: 32.0
```

```
GoalDist.scale: 24.0
```

```
RotateToGoal.scale: 32.0
```

```
RotateToGoal.slowing_factor: 5.0
```

```
RotateToGoal.lookahead_time: -1.0
```



controller_params.yaml

local_costmap:

local_costmap:

ros__parameters:

update_frequency: 5.0

publish_frequency: 2.0

global_frame: odom

robot_base_frame: twil_origin

rolling_window: true

width: 3

height: 3

resolution: 0.05

robot_radius: 0.3

plugins: ["inflation_layer"]



controller_params.yaml

inflation_layer:

 plugin: "nav2_costmap_2d::InflationLayer"

 cost_scaling_factor: 3.0

 inflation_radius: 0.35

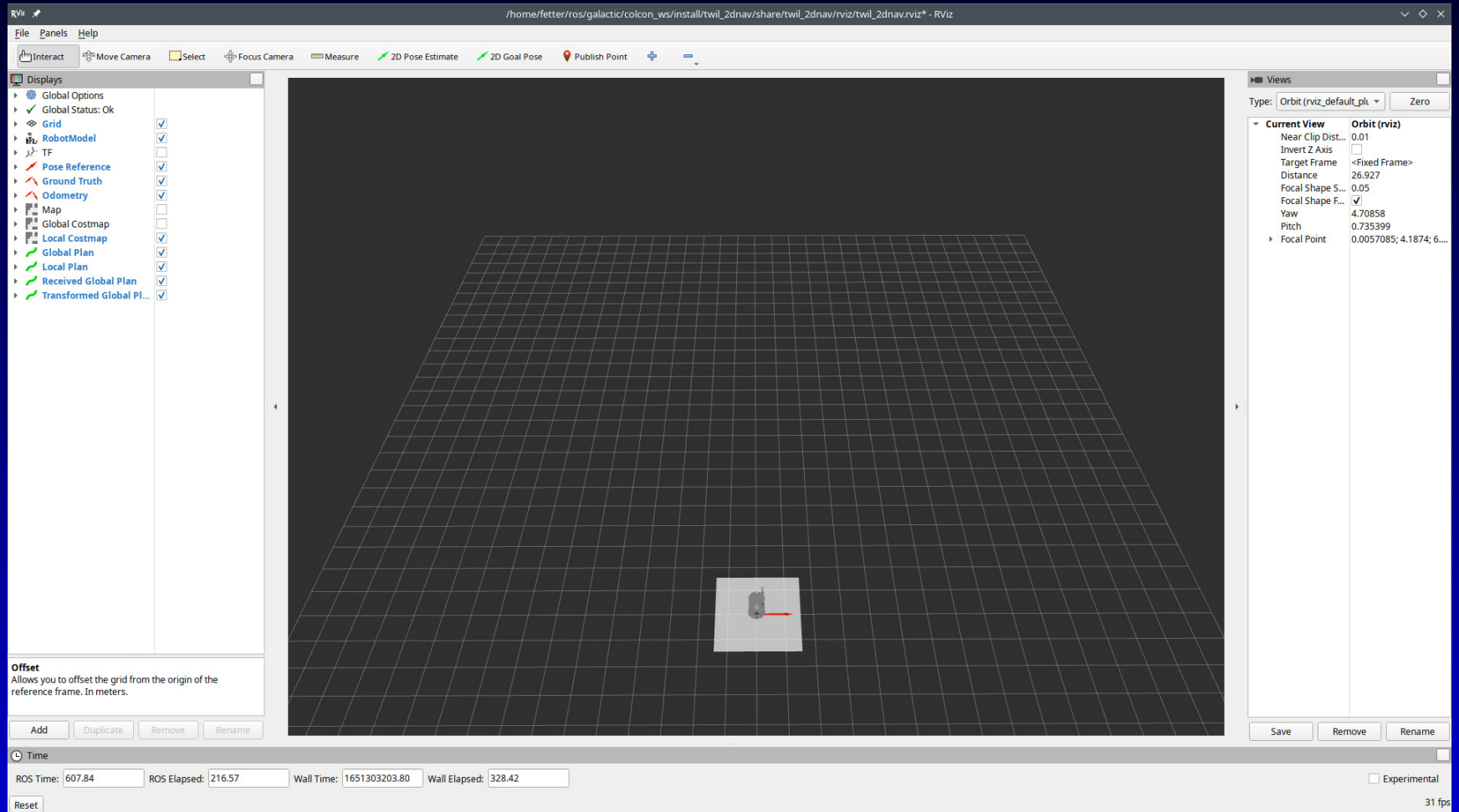
static_layer:

 map_subscribe_transient_local: True

always_send_full_costmap: True



Mapa de Custo Local



Árvores de Comportamento

- Representam as tarefas a serem completadas
- Nodo envia um *tick* para o 1º filho
 - Se o filho retorna sucesso, o nodo envia um *tick* para o próximo filho
 - Se o filho retorna falha, o nodo retorna falha
 - Se todos filhos retornam sucesso, o nodo retorna sucesso
- Nodos intermediários representam condições
- Folhas representam ações
- Primitivas básicas podem ser criadas e reutilizadas para compor comportamentos



bt_navigator

- Nodos são implementas por *plugins*
- Utiliza a biblioteca `BehaviourTree.CPP`
 - Árvores representadas em XML
 - Pode carregar sub-árvores
 - Toda a árvore do `bt_navigator` pode ser carregada em uma árvore de mais alto nível
 - Navegação pode ser um ramo de uma árvore de uma tarefa de mais alto nível



bt_navigator_params.yaml

bt_navigator:

 ros__parameters:

 global_frame: map

 robot_base_frame: twil_origin

 odom_topic: /twist_mrac_linearizing_controller/odom

 bt_loop_duration: 10

 default_server_timeout: 20

 enable_groot_monitoring: True

 groot_zmq_publisher_port: 1666

 groot_zmq_server_port: 1667



bt_navigator_params.yaml

plugin_lib_names:

- nav2_compute_path_to_pose_action_bt_node
 - nav2_compute_path_through_poses_action_bt_node
 - nav2_follow_path_action_bt_node
 - nav2_back_up_action_bt_node
 - nav2_spin_action_bt_node
 - nav2_wait_action_bt_node
 - nav2_clear_costmap_service_bt_node
 - nav2_is_stuck_condition_bt_node
 - nav2_goal_reached_condition_bt_node
 - nav2_goal_updated_condition_bt_node
 - nav2_initial_pose_received_condition_bt_node
 - nav2_reinitialize_global_localization_service_bt_node
 - nav2_rate_controller_bt_node
 - nav2_distance_controller_bt_node
 - nav2_speed_controller_bt_node
-



bt_navigator_params.yaml

- nav2_truncate_path_action_bt_node
 - nav2_goal_updater_node_bt_node
 - nav2_recovery_node_bt_node
 - nav2_pipeline_sequence_bt_node
 - nav2_round_robin_node_bt_node
 - nav2_transform_available_condition_bt_node
 - nav2_time_expired_condition_bt_node
 - nav2_distance_traveled_condition_bt_node
 - nav2_single_trigger_bt_node
 - nav2_is_battery_low_condition_bt_node
 - nav2_navigate_through_poses_action_bt_node
 - nav2_navigate_to_pose_action_bt_node
 - nav2_remove_passed_goals_action_bt_node
 - nav2_planner_selector_bt_node
 - nav2_controller_selector_bt_node
 - nav2_goal_checker_selector_bt_node
-

Árvores de Comportamento

- `navigate_w_replanning_time.xml`
 - Replaneja o caminho global à 1 Hz
- `navigate_w_replanning_distance.xml`
 - Replaneja o caminho global periodicamente a cada 1 m
- `navigate_w_replanning_speed.xml`
 - Replaneja o caminho global periodicamente proporcional à velocidade
- `follow_point.xml`
 - Segue uma pose dinâmica a uma certa distância

Árvores de Comportamento

- `navigate_to_pose_w_replanning_and_recovery.xml`
 - Replaneja o caminho global à 1 Hz
 - Ações de recuperação para planejamento e controle
- `navigate_through_poses_w_replanning_and_recovery.xml`
 - Planeja o caminho global passando por uma sequência de pontos
 - Replaneja à 1 Hz
 - Ações de recuperação para planejamento e controle



navigate_w_replanning_time.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithReplanning">
      <RateController hz="1.0">
        <ComputePathToPose goal="{goal}" path="{path}" planner_id="
          GridBased"/>
      </RateController>
      <FollowPath path="{path}" controller_id="FollowPath"/>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

navigate_w_replanning_distance.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithReplanning">
      <DistanceController distance="1.0">
        <ComputePathToPose goal="{goal}" path="{path}" planner_id="
          GridBased"/>
      </DistanceController>
      <FollowPath path="{path}" controller_id="FollowPath"/>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

navigate_w_replanning_speed.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithReplanning">
      <SpeedController min_rate="0.1" max_rate="1.0" min_speed="0.0"
        max_speed="0.26" filter_duration="0.3">
        <ComputePathToPose goal="{goal}" path="{path}" planner_id="
          GridBased"/>
      </SpeedController>
      <FollowPath path="{path}" controller_id="FollowPath"/>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

follow_point.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithReplanning">
      <RateController hz="1.0">
        <Sequence>
          <GoalUpdater input_goal="{goal}" output_goal="{updated_goal}">
            <ComputePathToPose goal="{updated_goal}" path="{path}"
planner_id="GridBased"/>
          </GoalUpdater>
          <TruncatePath distance="1.0" input_path="{path}" output_path="{
truncated_path}"/>
        </Sequence>
      </RateController>
      <KeepRunningUntilFailure>
        <FollowPath path="{truncated_path}" controller_id="FollowPath"/>
      </KeepRunningUntilFailure>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

navigate_to_pose_w_replanning_and_recovery.xml

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <RecoveryNode number_of_retries="6" name="NavigateRecovery">
      <PipelineSequence name="NavigateWithReplanning">
        <RateController hz="1.0">
          <RecoveryNode number_of_retries="1" name="ComputePathToPose">
            <ComputePathToPose goal="{goal}" path="{path}" planner_id="GridBased"/>
            <ReactiveFallback name="ComputePathToPoseRecoveryFallback">
              <GoalUpdated/>
              <ClearEntireCostmap name="ClearGlobalCostmap-Context" service_name="
global_costmap/clear_entirely_global_costmap"/>
            </ReactiveFallback>
          </RecoveryNode>
        </RateController>
        <RecoveryNode number_of_retries="1" name="FollowPath">
          <FollowPath path="{path}" controller_id="FollowPath"/>
          <ReactiveFallback name="FollowPathRecoveryFallback">
            <GoalUpdated/>
            <ClearEntireCostmap name="ClearLocalCostmap-Context" service_name="
local_costmap/clear_entirely_local_costmap"/>
          </ReactiveFallback>
        </RecoveryNode>
      </PipelineSequence>
    </RecoveryNode>
  </BehaviorTree>
</root>
```



navigate_to_pose_w_replanning_and_recovery.xml

```
</RecoveryNode>
</PipelineSequence>
<ReactiveFallback name="RecoveryFallback">
  <GoalUpdated/>
  <RoundRobin name="RecoveryActions">
    <Sequence name="ClearingActions">
      <ClearEntireCostmap name="ClearLocalCostmap-Subtree" service_name="
local_costmap/clear_entirely_local_costmap"/>
      <ClearEntireCostmap name="ClearGlobalCostmap-Subtree" service_name="
global_costmap/clear_entirely_global_costmap"/>
    </Sequence>
    <Spin spin_dist="1.57"/>
    <Wait wait_duration="5"/>
    <BackUp backup_dist="0.15" backup_speed="0.025"/>
  </RoundRobin>
</ReactiveFallback>
</RecoveryNode>
</BehaviorTree>
</root>
```



navigate_through_poses_w_replanning_and_recovery.xml

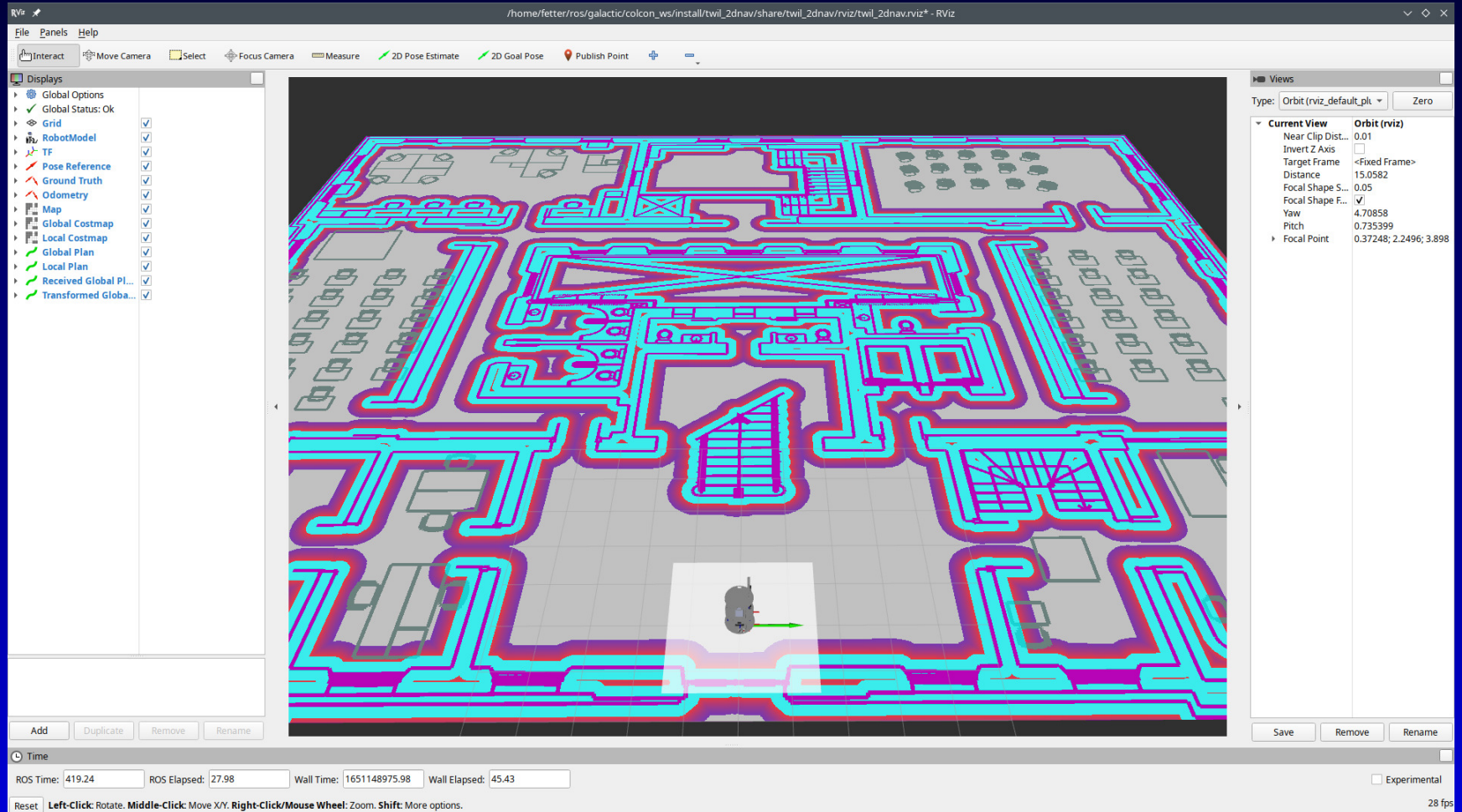
```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <RecoveryNode number_of_retries="6" name="NavigateRecovery">
      <PipelineSequence name="NavigateWithReplanning">
        <RateController hz="0.333">
          <RecoveryNode number_of_retries="1" name="ComputePathThroughPoses">
            <ReactiveSequence>
              <RemovePassedGoals input_goals="{goals}" output_goals="{goals}" radius="0.7"/>
              <ComputePathThroughPoses goals="{goals}" path="{path}" planner_id="GridBased"/>
            </ReactiveSequence>
            <ReactiveFallback name="ComputePathThroughPosesRecoveryFallback">
              <GoalUpdated/>
              <ClearEntireCostmap name="ClearGlobalCostmap-Context" service_name="
global_costmap/clear_entirely_global_costmap"/>
            </ReactiveFallback>
          </RecoveryNode>
        </RateController>
        <RecoveryNode number_of_retries="1" name="FollowPath">
          <FollowPath path="{path}" controller_id="FollowPath"/>
          <ReactiveFallback name="FollowPathRecoveryFallback">
            <GoalUpdated/>
          </ReactiveFallback>
        </RecoveryNode>
      </PipelineSequence>
    </RecoveryNode>
  </BehaviorTree>
</root>
```

```

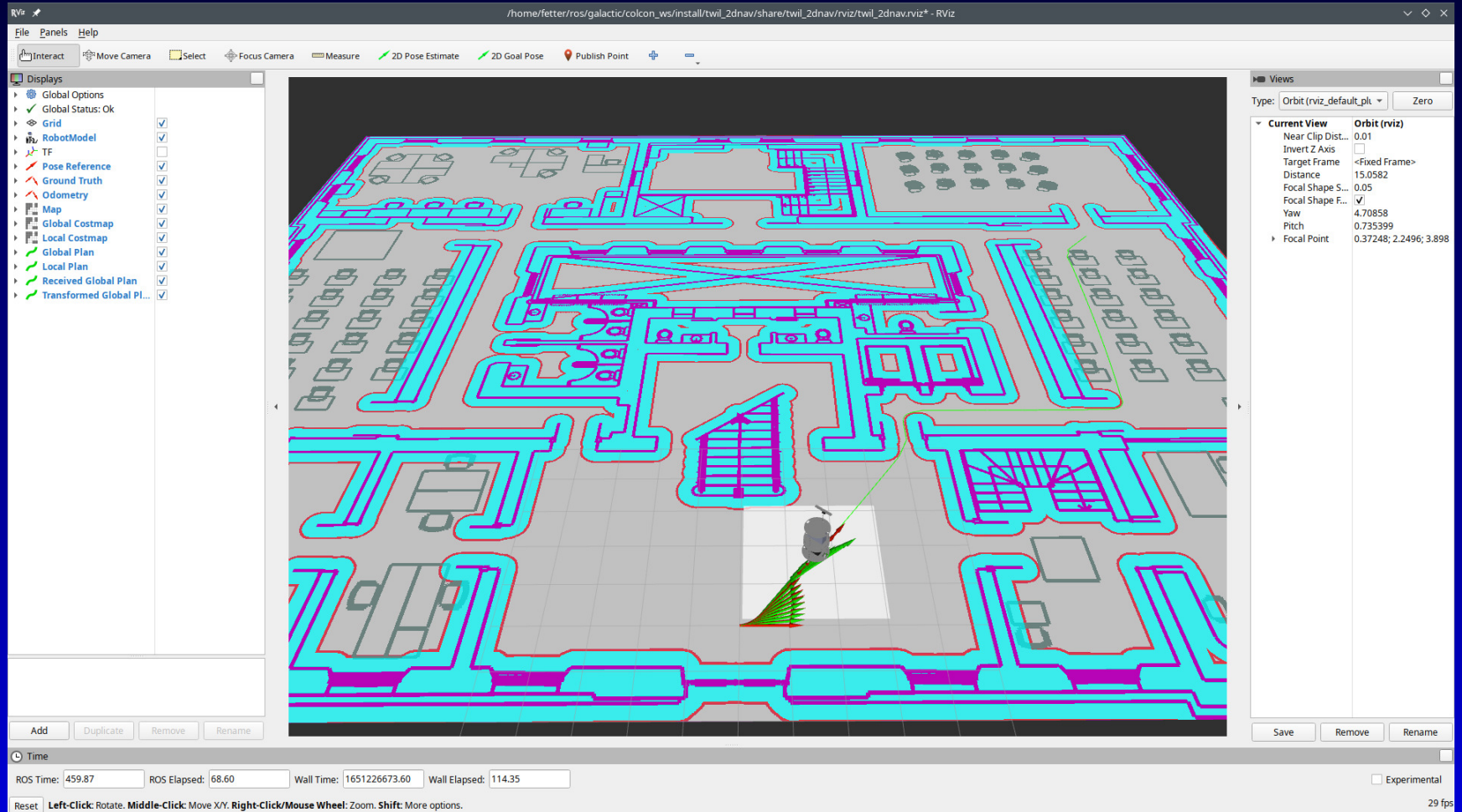
    <ClearEntireCostmap name="ClearLocalCostmap-Context" service_name="
    local_costmap/clear_entirely_local_costmap"/>
  </ReactiveFallback>
</RecoveryNode>
</PipelineSequence>
<ReactiveFallback name="RecoveryFallback">
  <GoalUpdated/>
  <RoundRobin name="RecoveryActions">
    <Sequence name="ClearingActions">
      <ClearEntireCostmap name="ClearLocalCostmap-Subtree" service_name="
      local_costmap/clear_entirely_local_costmap"/>
      <ClearEntireCostmap name="ClearGlobalCostmap-Subtree" service_name="
      global_costmap/clear_entirely_global_costmap"/>
    </Sequence>
    <Spin spin_dist="1.57"/>
    <Wait wait_duration="5"/>
    <BackUp backup_dist="0.15" backup_speed="0.025"/>
  </RoundRobin>
</ReactiveFallback>
</RecoveryNode>
</BehaviorTree>
</root>

```

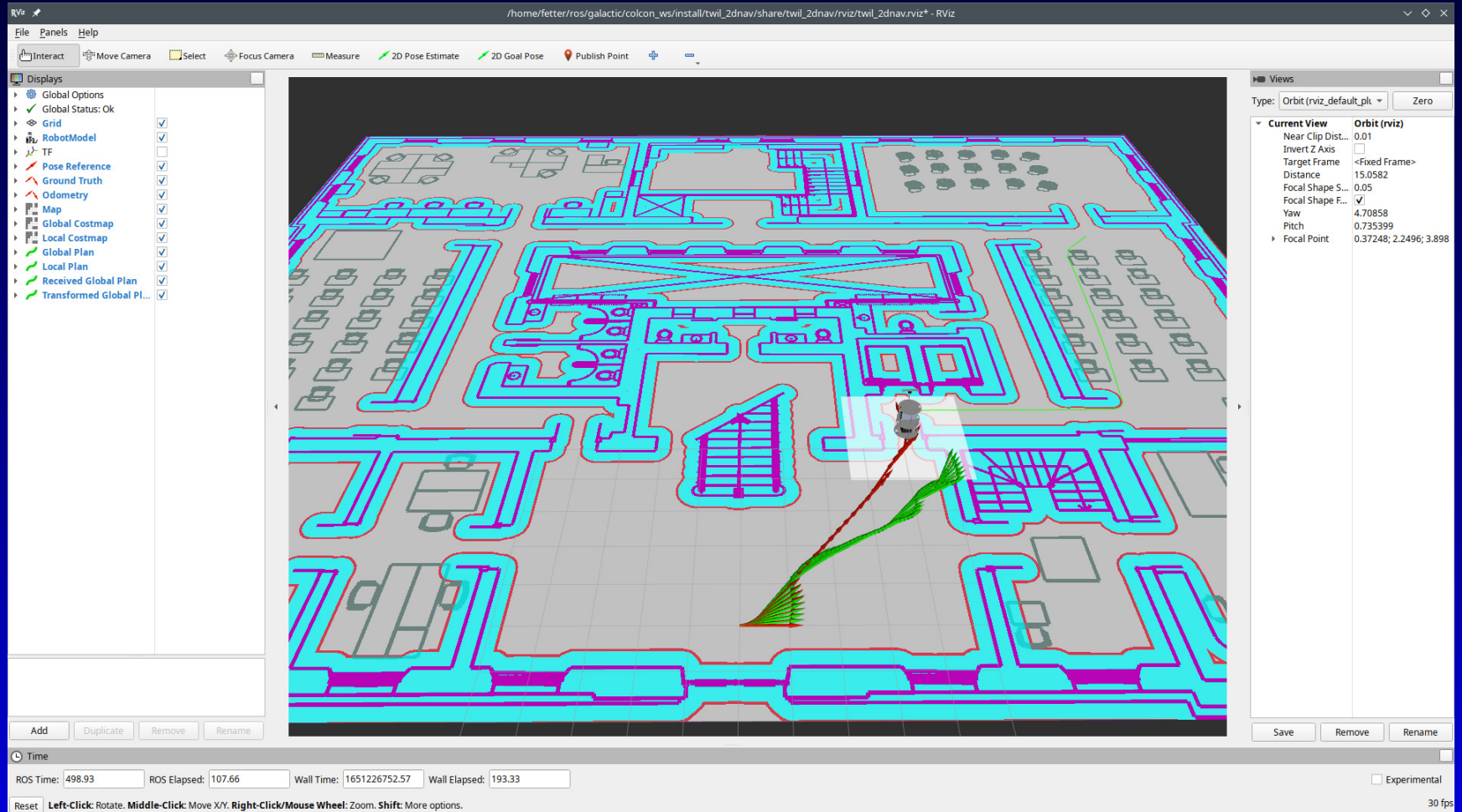
Trajeto ria no RViz



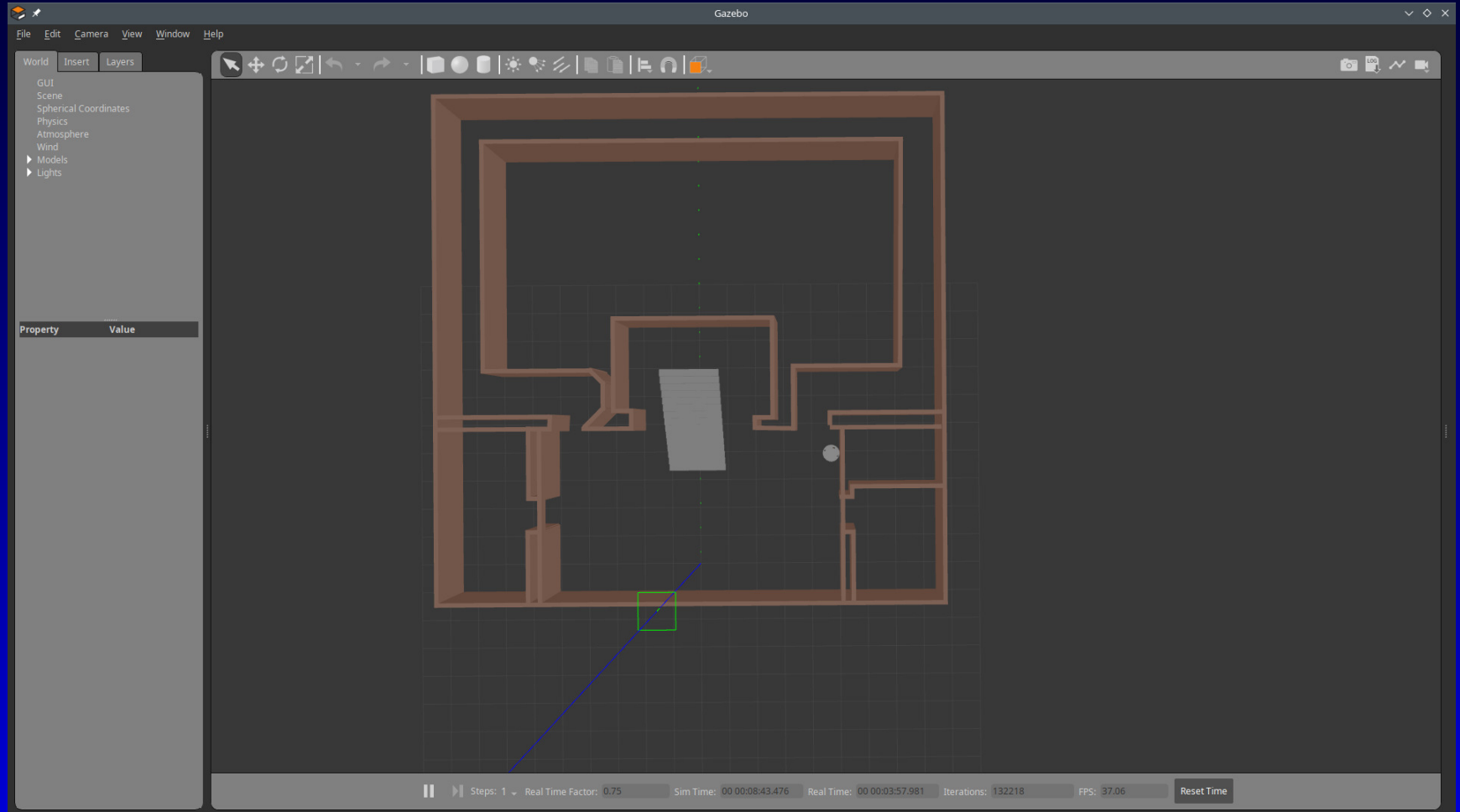
Trajetória no RViz



Trajeto ria no RViz



Trajetória no Gazebo



- O erro da odometria faz o robô bater na parede

Instalação do `twil_2dnav`

- Instalação inicial

```
cd ~/colcon_ws/src
```

```
git clone -b $ROS_DISTRO http://git.ece.ufrgs.br/cca99006/twil
```

```
touch twil/twil_ident/COLCON_IGNORE
```

```
touch twil/twil_gazebo_ros_control/COLCON_IGNORE
```

```
cd ~/colcon_ws
```

```
colcon build --symlink-install
```

```
source ~/colcon_ws/install/setup.bash
```

- Atualização

```
cd ~/colcon_ws/src/twil
```

```
git pull
```

```
cd ~/colcon_ws ; colcon build --symlink-install
```

```
source ~/colcon_ws/install/setup.bash
```

Instalação do `ufrgs_gazebo`

- Instalação inicial

```
cd ~/colcon_ws/src
```

```
git clone -b $ROS_DISTRO http://git.ece.ufrgs.br/cca99006/  
ufrgs_gazebo
```

```
cd ~/colcon_ws
```

```
colcon build --symlink-install
```

```
source ~/colcon_ws/install/setup.bash
```

- Atualização

```
cd ~/colcon_ws/src/ufrgs_gazebo
```

```
git pull
```

```
cd ~/colcon_ws ; colcon build --symlink-install
```

```
source ~/colcon_ws/install/setup.bash
```

Instalação do `ufrgs_maps`

- Instalação inicial

cd ~/colcon_ws/src

git clone -b \$ROS_DISTRO http://git.ece.ufrgs.br/cca99006/ufrgs_maps

cd ~/colcon_ws

colcon build --symlink-install

source ~/colcon_ws/install/setup.bash

- Atualização

cd ~/colcon_ws/src/ufrgs_maps

git pull

cd ~/colcon_ws ; **colcon** build --symlink-install

source ~/colcon_ws/install/setup.bash

Mover o Robô

ros2 launch twil_2dnav gazebo.launch.xml pause:=**false**

- Pegar no RViz a ferramenta *2D Goal Pose* e marcar a pose de destino

Gráfico de Computação

rqt_graph &

